

Fuel Prices Dashboard

Name:	Adnan Ali, Akarsh Kumar, Akshat Jain, Annie Shorya, Kevin Ninan Mathew
Student Number:	540372210, 540892291, 540775374, 540540853, 540489161

1 Introduction

Using live data from the NSW Government's FuelCheck API, this report shows how a real-time fuel price monitoring system was built. We cover five main components for this project: data retrieval, integration and cleaning, publishing via MQTT, subscription to a local broker, and visualization through an interactive Streamlit dashboard. Authenticated API calls were used to get live data, which was then processed to make a single dataset by combining fuel prices and station information. Each record was sent to MQTT topic at regular intervals as a JSON message after cleaning and filtering invalid entries. A background MQTT subscriber received these messages, stored them without duplication, and updated a reusable data file. After the data was put together, it was shown on a dynamic map interface that let users explore fuel prices by station and type of fuel. This live data pipeline successfully shows methods for real-time processing, message streaming, and designing a user-centric dashboard.

2 Data Retrieval

The initial step in our pipeline was the retrieval of live fuel pricing data from the official NSW Fuel API.

2.1 API Source and Authentication

The data is fetched from the Fuel API, utilizing its v1 endpoints. Access requires OAuth2 client credentials for authentication. We first obtained the access token using client credentials. The `SecurityToken()` function constructs a GET request to the API's token endpoint with the necessary authorization header. On receiving successful HTTP 200 response, the function parses the returned JSON to extract the `access_token`. The obtained token is stored in global variable (`ACCESS_TOKEN`), which is then used for subsequent data-fetching API calls. The function includes basic error handling to print status codes and response text in case where token retrieval fails.

2.2 Data Fetching

Using the access token, the `retrieve_data()` function is invoked to fetch the fuel station and pricing data from the API. This function uses the `/fuel/prices` endpoint of the `Base_URL`. It then generates a `request_timestamp` and the transaction counter (`CNT`) is incremented for each request. Required headers are constructed for the GET request, which includes the Authorization header, the Content-Type, the `apikey` (using the global `API.Key`), the generated `transactionid`, and the `request_timestamp`. The successful response (status 200) gives a JSON object containing two primary arrays: `stations` (with station details) and `prices` (fuel prices of various stations). These arrays are then returned by the function. When an API error occurs, empty list is return along with an error message.

3 Data Integration, Cleaning & Storage

After retrieving the raw data, we process it to create a consolidated and clean dataset which will be used for further analysis and visualisation.

3.1 Data Integration

The `transform_save()` function is used to integrate the station and price datasets. First, we create an efficient `station_lookup` dictionary, this dictionary is made by iterating through the `stations` list, and using unique code for each station as key, and storing a dictionary of its details like `brand_id`, `station_id`, `station_name`, `brand_name`, `latitutde` and `longitude`. Next, the function iterates through the `prices` list. For each price, it fetches the station code and uses this information in the `station_lookup` dictionary. If a match is found, new record is created by

combining the station's full details with the specific price of different fuel type and its last updated information. The resultant records are appended to a list, which is used to construct Pandas DataFrame, this represents the merged dataset.

3.2 Data Cleaning and Preprocessing

Exploratory Data Analysis (EDA): Before cleaning the data, we looked at it closely through (EDA). For example, a graph of the raw fuel prices (Figure 1) showed many prices at zero, which are clearly errors, helping us decide what to fix the in the cleaned dataset.

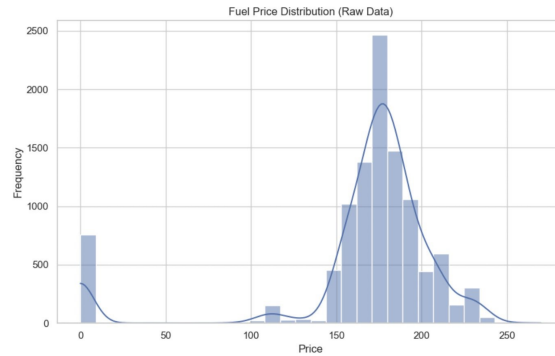


Figure 1: Histogram illustrating the frequency distribution of raw fuel prices from the API

Exploratory Data Analysis (EDA): Before cleaning the data, we looked at it closely through (EDA). For example, a graph of the raw fuel prices (Figure 1) showed many prices at zero, which are clearly errors, helping us decide what to fix the in the cleaned dataset.

3.2.1 Handling Missing Values

The rows with critical fields (stationid, address, latitude, longitude) which contain missing values are dropped, ensuring data integrity for essential attributes.

3.2.2 Data Type Conversion

Latitude and longitude columns are converted to numeric types using `pd.to_numeric`. The `errors='coerce'` argument ensures that values that cannot be converted to numbers become NaN (Not a Number). Furthermore, the rows containing NaN values are dropped. The `isAdBlueAvailable` column were converted to a boolean type.

3.2.3 Text Standardization

String fields like brand and name are standardized by removing whitespace and the brand names are converted to title case.

3.2.4 Geographical Filtering

The filter is applied to retain only station located within a geographical bounding box defined for New South Wales with latitude between -38 and -28, longitude between 140 and 154.

3.3 Storage

The `transform_save()` saves the DataFrame to `fuelPrice_data.csv` using `df.to_csv("fuelPrice_data.csv", index=False)`. The CSV file is overwritten during each cycle when new data is fetched from the API, ensuring that it always reflects the most recent data.

4 Data Publishing via MQTT

After data cleaning, the code publishes each processed record to an MQTT (Message Queuing Telemetry Transport) broker, enabling real-time data distribution to the dashboard.

4.1 MQTT Broker and Client Setup

The connection is established with the MQTT broker using the address (172.17.34.107) and port (1833). The `paho.mqtt.client` library is used to handle the message protocol. Inside the `publish_to_mqtt()` function, an MQTT client instance is created specifying `protocol=mqtt.MQTTv311`. This client then connects to the broker using a `keepalive` interval of 60 seconds, this ensures consistent communication during the publishing process.

4.2 Message Publishing

The `publish_to_mqtt()` function takes the cleaned DataFrame as input and iterates through each row, where each row represents the fuel price data for a specific station. The data is first converted into a JSON string format using the `row.to_json()` method and this serves as the message payload. Each message is then published to a predefined MQTT topic, specified by the `MQTT_topicconstant` (set to `fuel/check`), using the `client.publish()` method. To prevent overloading MQTT broker or the subscriber, a delay of 0.1 seconds is introduced between message publication using `time.sleep(0.1)`. After all records from the DataFrame is successfully published, a confirmation message is printed, and the MQTT client disconnects from the broker.

5 Data Subscription

The dashboard application actively subscribes to the designated MQTT topic to receive the real-time fuel price data.

5.1 MQTT Client in Dashboard

We initialised and managed its MQTT client within the `mqttSubscriber()` function. This function is designed to run in a separate background thread which initiates using threading. It ensures that the MQTT listening loop does not block the main Streamlit application thread that maintains UI responsiveness. The MQTT client Configuration which is Broker address, port and protocol version `MQTTc311` mirrors that of the producers. The Callback functions such as `onConnect`, `onSubscribe`, `onMessage`, `onDisconnect` are assigned to the client to handle different MQTT events. The `loop_forever()` method is called to keep the client connected and listening for messages. The function also includes a basic try-except block to attempt reconnection after a delay if errors occur.

5.2 Callback Handling

The `onConnect(client,userdata,flags,rc)` callback is triggered when the dashboard client successfully connects to the MQTT broker (indicated by `rc==0`). After getting connected this function immediately subscribes the client to the `mqtt_topic(fuel/check)` which ensures that it started receiving messages.

The `onMessage(client,userdate,msg)` callback is the core component for processing incoming data. When a message arrives on the subscribed topic this function is invoked. The message payload which is a JSON string is decoded. A unique key is generated for each price entry based on `stationid`, `fueltype` and `lastupdated` to help manage duplicates within the current session using an existing `key` set. We implemented a strategy that involves a global buffer and a `data.initialised` flag. Initially, incoming records are added to the buffer. Once this buffer reaches a predefined `initial_records_threshold` the `saveInitialData` function is called in a new thread to write the buffer's content to a JSON file which is `fuel_data.json` file. After this initial save, `data.initialised` is set to `true`. Then the messages are then processed by loading existing data from `fuel_data.json` appending the new unique entry and then resaving the entire updated dataset to `fuel_data.json` using the `saveData` function which employs a `FileLock` for concurrent access.

6 Data Visualisation

The dashboard we developed using Streamlit provides an interactive visualisation of the fuel prices data on a map generated by Folium library.

6.1 Dashboard Framework and Map Library

Streamlit serves as the web application framework, enabling rapid development of interactive UIs. Folium is leveraged for its capabilities in creating rich, interactive Leaflet maps, and `streamlit_folium` provides the component to embed these maps within the Streamlit application.

6.2 Dynamic Updates and Data Loading

Our dashboard shows live update mainly in 2 ways The `st.autorefresh` feature makes Streamlit rerun the script every 10 second. every time the script reruns, it calls the `loadDataForDisplay()` function. This function reads the fuel data from a file called `fuel_data.json`. This file is always being updated with new data by the MQTT system in the background. Because we keep loading and showing the data from this file, map always displays the latest information when it rerun.

6.3 Data Preparation for Map

When we load the data from json file, we get a list of price records. The `groupDataByStation()` function organises these records. It groups all the fuel prices and details like name, brand, address and coordinates for each station so that all the info for one station is together. This make it much easier to uuse the data for the map.

6.4 Map Creation and Marker Rendering

A base folium map `folium.map()` is initialised typically based on coordinates of filtered stations. The dashboard's `main()` function then iterates through the stations obtained from `groupDataByStation()` after applying user-selected filters for fuel type and brand. For each station that matches the active filters a `folium.Marker` is created and added to the map at the station's latitude and longitude. The popup activated on clicking a marker displays the stations name, address and the latest prices for all available fuel types along with the update time. Our aim was to show all the details of the fuel for the current selected fuel tupe and the stations lastupdated timestamp. All this is rendered in Streamlit interface using `st.folium`.

6.5 Interactivity (Filters)

Our dashboard has filter widgets located right in middle above the map.

A `st.selectbox` allows users to choose a specific fuel type such as U91, E10, with U91 often set as a default if available.

A `st.multiselect` enables users to filter the displayed stations by one or more brands. When a filter selection is changed, Streamlit automatically re-runs the script. The data is re-filtered based on the new selections, and the map is re-rendered to reflect the changes which provide an interactive data exploration experience.

7 Continuous Execution (Producer)

We have implimented the `run_service()` function which encompasses the sequence of data retrieval, processing that is transformation and cleaning and MQTT publishing later conludes with a `time.sleep(60)` statement. As The project's Task 5 specifies that the producer pipeline (Tasks 1-3) should operate continuously we have add a 60-second delay between each new round of API data retrieval. For true continuous operation, these steps would typically be enclosed witin an infinite while `True:loop` to ensure the cycle repeats indefinitely. The 0.1-second delay is mandated between publishing individual MQTT messages which we have implemented within the `publish_to_qtt()` function thatensures controlled message flow.

8 Challenges Overcome

8.1 API Token Management

When fetching data from the NSW FuelCheck API, we had to manage authentication tokens that were expiring continuously. To avoid interruptions, we wrote the code to detect token errors and automatically refreshing the token, make sure that our data pipeline runs continuously without intervention.

8.2 Data Cleaning and Integration

The raw API data was messy, with missing fields, inconsistent data types, and few duplicates. We wrote cleaning methods to drop incomplete rows, convert coordinates to numbers, and standardize text fields. This made our dataset reliable for both analysis and real-time use.

8.3 Efficient Data Publishing (MQTT)

To make a live data stream, we needed to publish messages at regular intervals without overloading the MQTT broker or dashboard. We added a short delay between messages and made each record sent once per session, keeping the data flow smooth and manageable.

8.4 Live Dashboard

While developing the live dashboard, we found that directly updating Streamlit's in-memory session state from the MQTT subscriber thread was unreliable. Sometimes, the dashboard wouldn't pick up new data in real time, likely due to how Streamlit manages state and threading. To solve this, we introduced a buffer using a JSON file as an intermediary. We chose JSON for the buffer because it's easy to write and read in Python, and it stores the nested fuel price records without needing extra flattening or conversion. The MQTT thread writes each unique record to the JSON file and the Streamlit dashboard simply reloads the latest data from this file at regular intervals.

8.5 Live threading

MQTT data is integrated with Streamlit with running MQTT client in background thread. This made the dashboard responsive, but there were data corruptions which we handled through thread locks. We also saw some "ScriptRunContext" warnings in the logs, but they were not affecting our result.

8.6 In-Memory Deduplication

Rather than constantly reading and writing to a JSON file we made records in memory on track by using unique keys. This helped us to quickly filter out duplicates and keep the dashboard durable and reliable.

8.7 User Experience

We added filters for fuel type and brand and set up the dashboard to auto-refresh in every 10 seconds, so the dashboard displays the latest prices without needing to reload the page.

9 Future Recommendations

To further enhance the efficiency and feature set of this FuelCheck data pipeline, the following improvements could be considered:

Producer Token Auto-Refresh: Implement an automatic token refresh to handle authentication failures. This ensures continuous API access without manual intervention, improving reliability.

Real-time dashboard updates: Refactor the dashboard code, so the `on_message` callback updates `st.session_state` directly. This allows the dashboard to re-render dynamically with new data, eliminating the need for intermediate file I/O.

Optimized Dashboard Data Handling: Use lightweight databases or incremental updates instead of full JSON file reads to improve dashboard performance and session persistence.

Improved logging: Use python logging module for better error tracking and monitoring, during background or long-running operations.

Advanced Mapping Features: Enhance the dashboard's map by adding marker clustering to improve performance and navigation when displaying many fuel stations.

Enhanced Security Practices: Avoid hardcoding sensitive information like API keys. Use environment variables or secret management tools for secure handling, especially in production environments.

10 Conclusion

In this assignment we built a real-time data pipeline for NSW fuel prices. Firstly, we wrote the code that retrieves live data from the NSW FuelCheck API, cleans it up, and saves it as a CSV file. Then, each fuel price record is published as a message to an MQTT broker, making a live data stream.

On the dashboard, we used Streamlit to create an interactive web app. The dashboard receives MQTT messages, updates a local JSON file as a buffer, and displays all the latest station prices on a map. We can filter by fuel type or brand and click on any station to see more details. The dashboard refreshes automatically,

so users always see latest and updated prices.

Throughout the assignment, we did key data engineering parts which includes connecting to APIs, cleaning and merging data, handling real-time data, preventing duplicates, and visualizing everything in a dashboard. While our system meets the assignment's main goal, but in future we can improvise by implementing things like more better error handling and make faster updates.